

Canal de handshake pai-filho em um runtime estilo Docker construído em Python, Cython e C++

Pesquisa aprofundada sobre a distinção entre sucesso de bootstrap e falha de `execve`, com pipe `CLOEXEC` e mensagens por stage

Relatório técnico em português

Data de geração: 12 de março de 2026

Escopo: Linux/OCI, runtime estilo Docker, control plane em Python, data plane em C++, cola em Cython

Tese central: use dois canais complementares. Um `pipe2(0_CLOEXEC)` dedicado resolve a barreira final de `execve()`. Um `socketpair(AF_UNIX, SOCK_SEQPACKET|SOCK_CLOEXEC)` resolve mensagens por stage, ACKs e passagem de file descriptors.

1. Resumo executivo

Este relatório defende que a feature pedida não deve ser tratada como um detalhe periférico de IPC, mas como um contrato semântico central do runtime. Em um launcher de containers, ``fork()`` ou ``clone()`` apenas provam que um processo filho existe; eles não provam que namespaces, rootfs, cgroups, credenciais, seccomp e hooks foram aplicados corretamente, nem que o binário final foi carregado com sucesso por ``execve()``. A decisão arquitetural correta é separar duas perguntas que uma implementação ingênua costuma misturar. A primeira é se o bootstrap do processo de init chegou a um ponto consistente, observável e autorizável pelo pai. A segunda é se a fronteira final de ``execve()`` foi de fato cruzada, isto é, se o kernel substituiu a imagem do processo pelo payload do container.

Para responder à segunda pergunta com semântica forte, a primitiva mais confiável continua sendo um ``pipe2(O_CLOEXEC)`` dedicado ao erro de ``execve()``. O filho mantém a ponta de escrita aberta durante o bootstrap; em um ``execve()`` bem-sucedido, o kernel fecha esse descritor automaticamente por causa de ``FD_CLOEXEC``; se ``execve()`` ou qualquer passo terminal pré-exec falhar, o filho escreve um frame de erro estruturado no pipe e encerra o processo. Para responder à primeira pergunta, porém, o pipe sozinho é insuficiente. O projeto precisa de um segundo canal, idealmente um ``AF_UNIX SOCK_SEQPACKET | SOCK_CLOEXEC``, para mensagens por stage, ACKs, requests dirigidos ao pai e, quando necessário, passagem de file descriptors por ``SCM_RIGHTS``. Essa composição reproduz a lição prática do Docker original, de containerd e de runc: a barreira final de ``exec`` pede semântica binária e confiável; o bootstrap inteiro pede um protocolo mais rico, versionado e observável.

2. Introdução e recorte arquitetural

O contexto assumido neste relatório é um runtime Linux compatível com a lógica de um sistema estilo Docker, mas implementado com separação explícita de camadas: Python para control plane, metadados e API; C++ para data plane, syscalls, abstrações de kernel e interação com hardware; Cython como ponte tipada entre os dois mundos. Esse recorte é coerente com a realidade operacional de runtimes de containers. O control plane precisa de elasticidade, introspecção, ergonomia de linguagem e integração com API, banco e fila de eventos; o data plane precisa de previsibilidade de ABI, controle fino sobre descritores, baixo overhead de syscall, facilidade para usar RAII e evitar custo do interpretador em caminhos críticos.

A feature pedida, “canal de handshake pai-filho — distinguir sucesso de bootstrap de falha de ``execve()``; pipe ``CLOEXEC``, mensagens por stage”, vive exatamente na linha divisória entre essas camadas. Ela nasce como uma preocupação de kernel, mas o impacto real aparece no domínio de alto nível. Um erro nessa feature faz o control plane registrar estados incorretos, vaza descritores para o payload, oculta a

causa real de falhas de start, quebra semântica de ``create`` e ``start``, e complica tanto rootless quanto segurança. Por isso, a solução não pode ser apenas “um pipe para stderr” ou “um retorno numérico de processo”. Ela precisa carregar semântica explícita de ciclo de vida.

A metodologia usada aqui combina leitura da especificação OCI de runtime, documentação oficial de Docker e Python, documentação do Cython e, principalmente, análise da implementação real de ``runc`/`libcontainer``, que é o ponto em que o Docker original materializa esse problema. O objetivo não é copiar mecanicamente o código em Go/C de ``runc``, mas extrair seus princípios arquiteturais e traduzi-los para uma solução idiomática em Python, Cython e C++.

3. Conceitos fundamentais

A expressão “handshake pai-filho” designa, neste contexto, o protocolo privado pelo qual o processo pai que lança o container e o processo filho responsável pelo bootstrap trocam estados, permissões para avançar, diagnósticos de falha e, em alguns casos, descritores adicionais. O termo “pai” aqui não é meramente genealógico; ele representa a autoridade que ainda detém contexto do host, metadados do container e responsabilidade por persistir a transição de estado. O termo “filho” também não é apenas o processo recém-criado; ele é o agente que vai entrar em namespaces, ajustar cgroups, montar rootfs, aplicar credenciais, instalar seccomp e, ao final, tentar o ``execve()`` da carga útil. Em um runtime de containers, o handshake existe porque a criação do processo não coincide com o início válido do container.

“Bootstrap” é o nome correto para essa sequência intermediária entre a criação do processo e a execução do programa alvo. Em um container runtime, bootstrap inclui pelo menos a criação ou entrada em namespaces, a configuração de cgroups, a preparação da root filesystem, a sanitização de file descriptors, a aplicação de credenciais e capabilities, a eventual execução de hooks OCI e a instalação de políticas de segurança. Em ambientes rootless ou com ``usersns``, bootstrap pode incluir um diálogo adicional com o pai para escrever ``/proc/<pid>/uid_map``, ``/proc/<pid>/gid_map`` e, dependendo do caso, ``setgroups``. Em implementações que seguem o ciclo OCI de ``create`` e ``start``, o bootstrap se divide em uma fase que torna o container “criado” e uma fase posterior que autoriza o ``exec`` do processo principal.

``execve()`` merece ser tratado como fronteira semântica própria. Em Linux, um ``execve()`` bem-sucedido não retorna; em caso de erro, ele retorna ``-1`` e ajusta ``errno``. Isso parece trivial, mas é o fundamento inteiro do problema. Se o pai só observa o código de saída do filho, ele não consegue saber se o filho morreu antes de carregar o binário ou se o binário foi carregado e a aplicação saiu imediatamente. Um processo que retorna ``127`` pode significar “arquivo não

encontrado” durante bootstrap, mas a própria aplicação dentro do container também pode terminar com `127` logo após um `exec` bem-sucedido. O handshake precisa quebrar essa ambiguidade.

`FD\_CLOEXEC` e `O\_CLOEXEC` são a pedra angular da barreira de `exec`. Um descritor marcado com close-on-exec é fechado automaticamente em um `execve()` bem-sucedido; se `execve()` falha, o descritor permanece aberto. O detalhe operacional importante é que a marcação deve ser atômica sempre que possível. Criar um pipe com `pipe()` e depois chamar `fcntl(F\_SETFD, FD\_CLOEXEC)` abre uma janela de corrida em programas multi-threaded: outra thread pode fazer `fork()` ou `execve()` antes da marcação e vaziar o descritor. Por isso, a primitiva recomendada é `pipe2(O\_CLOEXEC)` ou, no caso de sockets, `socketpair(..., SOCK\_CLOEXEC, ...)` . No universo Python esse detalhe é ainda mais relevante, porque o control plane tende a ser concorrente e a própria evolução da linguagem foi na direção de descritores não herdáveis por padrão.

“Mensagens por stage” significa tratar o bootstrap como máquina de estados observável, e não como bloco monolítico. Em vez de um booleano “deu certo / não deu certo”, o filho emite eventos como “user namespace precisa de mapeamento”, “hooks podem ser executados”, “rootfs preparada”, “pronto para iniciar”, “pronto para `exec`” ou “falha em seccomp com `errno=EPERM`”. Esse modelo é superior por três razões. Primeiro, ele dá ao pai contexto suficiente para tomar ações intermediárias, como escrever mapas de usuário ou passar um file descriptor. Segundo, ele produz rastreabilidade operacional, essencial para debugging e auditoria. Terceiro, ele separa etapas de bootstrap de semânticas de prontidão da aplicação, evitando que o runtime prometa mais do que o kernel realmente garantiu.

Primitiva	Fronteiras de mensagem	Passagem de FDs	Semântica de exec	Uso recomendado
pipe2(O_CLOEXEC)	Não preserva mensagens; fluxo unidirecional	Não	EOF em exec bem-sucedido; frame em falha	Barreira final de exec
socketpair(AF_UNIX, SOCK_STREAM SOCK_CLOEXEC)	Não preserva mensagens	Sim, via SCM_RIGHTS	Possível, mas exige framing extra	Console, control e simples, legado
socketpair(AF_UNIX, SOCK_SEQPACKET SOCK_CLOEXEC)	Preserva mensagens	Sim, via SCM_RIGHTS	Pode coexistir com a barreira; bom para estados	Mensagens por stage, ACKs e requests

Tabela 1 — Comparação entre primitivas de canal para um runtime de containers.

4. Contexto no Docker original, OCI e runc

A arquitetura original do Docker já embute a separação de responsabilidades que o seu projeto deseja reproduzir. A documentação oficial descreve Docker como uma arquitetura cliente-servidor, em que o cliente conversa com o daemon por API e o daemon gerencia objetos como imagens, containers, redes e volumes. Em versões modernas, boa parte do ciclo de vida do container passa por ``containerd``, enquanto a criação efetiva do ambiente OCI é delegada a um runtime como ``runc``. Isso importa porque o handshake pai-filho não acontece no nível da API REST do Docker; ele acontece vários níveis abaixo, no runtime de baixo nível que conversa diretamente com o kernel.

A especificação OCI ajuda a entender a semântica correta. Em OCI, ``create`` deve criar o ambiente de runtime do container, mas não deve executar o programa do usuário; ``start`` é a operação que efetivamente inicia o processo especificado. Essa distinção não é cosmética. Ela impõe a necessidade de uma barreira: o runtime precisa chegar a um estado intermediário consistente, no qual namespaces, mounts e demais recursos já foram preparados, mas o payload ainda não foi executado. O handshake é exatamente o mecanismo que torna essa fronteira visível e controlável.

Em ``runc``, o desenho concreto desse handshake mostra por que a sua feature não deve ser reduzida a um único pipe. O código de ``libcontainer`` cria mais de um canal interno. Há um socket par para inicialização, um socket de sincronização e um pipe de logs. No canal de sincronização aparecem mensagens como ``procError``, ``procReady``, ``procRun``, ``procHooks``, ``procHooksDone``, ``procMountPlease``, ``procMountFd``, ``procSeccomp`` e ``procSeccompDone``. O pai não apenas “espera o filho terminar”; ele participa ativamente do bootstrap, responde a pedidos do filho e decide quando o processo pode avançar. O uso de ``SOCK_SEQPACKET`` | ``SOCK_CLOEXEC`` no canal de sync ilustra uma lição importante: fronteiras de mensagem e higiene de descritores são requisitos de projeto, não refinamentos posteriores.

O arquivo ``nsexec.c`` de ``runc`` aprofunda ainda mais esse ponto. O bootstrap de namespaces não é um salto único, mas uma dança entre estágios. Existe um estágio inicial no pai, um estágio intermediário no primeiro filho e, em certas configurações, um estágio final no processo que se tornará o init do container. Esse arranjo existe porque certas operações, como escrita de ``uid_map`` e ``gid_map``, dependem de contexto do pai; outras precisam ocorrer já dentro de namespaces específicos; e a criação do processo final em um novo PID namespace também impõe ordem rígida. O resultado é um protocolo explícito de ``request`` e ``ack``, não um fluxo implícito baseado em timeout.

A grande lição ao contextualizar a feature no Docker original é que o runtime maduro separa responsabilidades por semântica. Logs não substituem canal de controle. Código de saída não substitui erro estruturado. EOF por si só não

substitui máquina de estados. E a higiene de file descriptors é tratada como propriedade de segurança. O seu projeto deve preservar exatamente essa filosofia, ainda que a implementação concreta use C++ em vez de Go e Cython como ponte para o Python.

runc/libcontainer	Significado operacional	Equivalente proposto
procReady	Filho terminou setup e aguarda liberação	READY_TO_START ou READY_FOR_EXEC
procRun	Pai autoriza continuação / execução	AUTHORIZE_EXEC
procHooks / procHooksDone	Sincronização de hooks OCI	HOOKS_REQUEST / HOOKS_ACK
procMountPlease / procMountFd	Pai abre ou encaminha mount FD	MOUNTFD_REQUEST / MOUNTFD_ACK
procSeccomp / procSeccompDone	Coordenação de seccomp listener	SECCOMP_REQUEST / SECCOMP_ACK
procError	Erro estruturado de init/bootstrap	BOOTSTRAP_ERROR

Tabela 2 — Leitura conceitual das mensagens de `runc` e equivalentes propostos.

5. Por que a feature é necessária no projeto proposto

A primeira razão para implementar essa feature é evitar um erro de modelagem muito comum: confundir existência de PID com sucesso de start. Um `fork()` bem-sucedido apenas significa que o kernel conseguiu duplicar o processo. Em um runtime de containers, ainda falta a parte cara: preparar namespaces, rootfs, cgroups, credenciais, hooks e segurança. Se o control plane persistir “container iniciado” antes de saber em que estágio o filho está, ele cria uma dívida operacional séria. O estado armazenado deixa de representar a realidade do kernel, e cada erro de start vira uma investigação forense baseada em logs e `strace`, quando deveria ser uma falha determinística com stage, `errno` e contexto.

A segunda razão é que nem código de saída nem `stderr` possuem semântica suficiente. O mesmo código numérico pode representar cenários radicalmente diferentes. O mesmo texto em `stderr` pode vir do launcher, de um hook OCI, do loader ELF ou da aplicação já executada. Misturar esses canais com um só fluxo textual destrói a capacidade do control plane de responder de forma correta. Para o usuário final, isso aparece como mensagens vagas do tipo “process exited with status 1”; para quem opera o sistema, aparece como a incapacidade de diferenciar um binário inexistente de um container que iniciou corretamente e falhou um milissegundo depois.

A terceira razão é específica da escolha por Python no control plane. A documentação do Python adverte que ``preexec_fn`` não é seguro na presença de threads, porque o filho pode deadlockar antes de ``exec``. Isso significa que uma solução “Python puro” que tente fazer o bootstrap diretamente no filho de ``subprocess`` é tecnicamente frágil. Somado a isso, desde o PEP 446 os descritores em Python são não herdáveis por padrão, o que é excelente para segurança, mas exige contratos explícitos quando um helper nativo precisa receber FDs. Em outras palavras: o handshake precisa existir e, além disso, deve morar em código nativo controlado, deixando ao Python o papel de orquestrar, persistir metadados e traduzir eventos de domínio.

A quarta razão é arquitetural. Um único pipe anônimo pode até servir para a barreira final de ``exec``, mas ele não resolve pedidos bidirecionais, não preserva fronteiras de mensagem, não passa FDs e não modela bem ``create`` separado de ``start``. É por isso que a solução correta é composta. O pipe ``CLOEXEC`` existe para responder à pergunta binária “o ``execve()`` cruzou a fronteira do kernel ou não?”. O canal de mensagens por stage existe para responder à pergunta processual “em que etapa do bootstrap o filho está, que ação do pai ele precisa e que diagnóstico estruturado deve ficar registrado?”.

6. Proposta de System Design

A macroarquitetura recomendada para o projeto tem quatro blocos. O primeiro é o control plane em Python, responsável por API, autorização, serialização da especificação do container, persistência de metadados, reconciliação de estado e exposição de eventos para o restante do sistema. O segundo é um adaptador em Cython, propositalmente fino, que faz a ponte tipada entre Python e o mundo nativo. O terceiro é um runtime worker nativo em C++, idealmente um processo separado ou um helper short-lived, responsável por toda a lógica que envolve ``clone``, ``fork``, ``execve``, namespaces, cgroups, mounts, credenciais, seccomp e handshake. O quarto bloco é o próprio processo de bootstrap, que começa como filho do worker e termina como init do container após o ``execve()`` do payload.

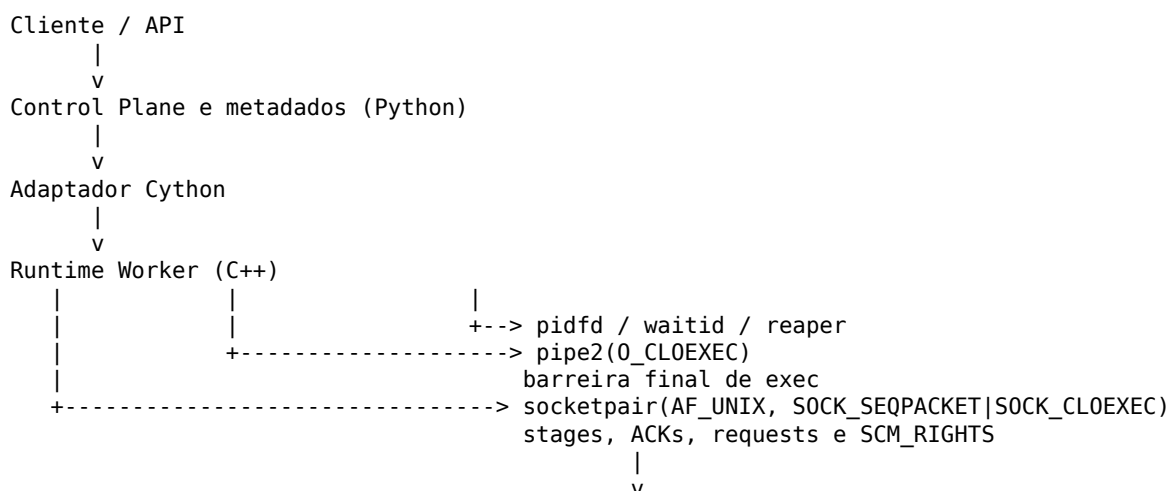
Do ponto de vista de robustez, o melhor desenho é evitar que o processo Python multi-threaded execute diretamente a sequência perigosa “``fork()`` e muita lógica antes de ``exec``”. Em vez disso, o Python deve invocar um worker nativo controlado, single-purpose, por meio de uma interface estreita. Essa invocação pode ocorrer por um client library em C++ exposto ao Python via Cython, e o worker em si pode ser um processo dedicado por launch ou um daemon nativo local acessado por socket Unix. O detalhe mais importante não é a topologia exata, e sim a regra de ouro: o caminho entre ``fork()``/``clone()`` e ``execve()`` deve permanecer no domínio nativo, com o mínimo possível de dependência do runtime Python.

No plano de ciclo de vida, a semântica recomendada replica a distinção OCI. A operação ``create`` deve terminar quando o bootstrap alcança um estágio consistente, por exemplo ``READY_TO_START``, em que o processo já está no contexto correto do container, mas ainda não executou o payload. A operação ``start`` deve enviar a autorização final para continuar e então aguardar a conclusão da barreira de ``exec``, observando o pipe ``CLOEXEC``. Com esse desenho, o sistema evita marcar o container como ``RUNNING`` cedo demais e também não confunde falha de ``execve()`` com encerramento rápido da aplicação.

O control plane deve persistir três visões complementares do mesmo fenômeno. A primeira é a intenção declarada pelo usuário ou pela API, isto é, o estado desejado. A segunda é a observação concreta do runtime, derivada do handshake, do ``pidfd`` e do reaper. A terceira é o transcript do bootstrap, que funciona como evidência histórica. Essa tripla separação facilita reconciliação, replays operacionais, diagnósticos e até auditoria, porque o sistema deixa de depender de heurísticas para explicar por que um container entrou ou não entrou em ``RUNNING``.

O Cython, nesse arranjo, não é o lugar para decidir política de bootstrap nem para realizar syscalls sensíveis; ele é a cola tipada. Sua responsabilidade é converter ``LaunchSpec`` de Python para estruturas nativas, chamar funções C++ com ``except +`` para tradução adequada de exceções, liberar o GIL em operações bloqueantes e devolver ao Python eventos de stage, handles, PIDs, ``pidfd`s` e diagnósticos estruturados. Em particular, o Cython deve evitar qualquer reentrada em Python no filho entre ``fork`` e ``exec``. Esse caminho crítico deve permanecer opaco para o interpretador.

Além da correção técnica, essa arquitetura dá ao control plane algo que o Docker real valoriza muito: uma trilha confiável de metadados. Cada tentativa de launch passa a ser uma entidade persistível, com ``container_id``, ``launch_attempt_id``, timestamps monotônicos de stage, PID do processo final, resultado da barreira de ``exec``, causa de falha, e eventual transcript completo. O resultado é um sistema que não depende de heurística para explicar por que um container não iniciou.



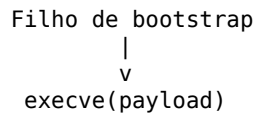


Figura textual 1 — Macroarquitetura recomendada para a divisão Python/Cython/C++.

Canal	Direção principal	Primitiva	Dono recomendado	Responsabilidade
Stage Sync	Bidirecional	AF_UNIX SOCK_SEQP CKET SOCK_CLOEXEC	Worker C++	Estados, requests, ACKs, passagem de FDs
Exec Barrier	Filho -> Pai	pipe2(O_CLOEXEC)	Worker C++	Distinguir sucesso de exec de falha terminal pré-exec
Log/Console	Filho -> observador	Pipe, PTY ou socket	Worker/console subsystem	Saída operacional, nunca semântica de controle
Reaper	Pai observa filho	pidfd + waitid, ou fallback	Worker C++	Classificar morte do processo sem corridas de PID
Spec/Command	Control plane -> worker	RPC local, UDS ou API nativa	Python + Cython + worker	Enviar LaunchSpec, start/kill/delete, coletar resultado

Tabela 3 — Canais recomendados e suas responsabilidades.

7. Integração Python, Cython e C++

A fronteira Python/Cython/C++ merece um desenho explícito, porque uma divisão ruim entre linguagens costuma anular a robustez do handshake. A camada Python deve operar com objetos de domínio estáveis, por exemplo `ContainerSpec`, `LaunchAttempt`, `StageEvent` e `BootstrapError`. A camada Cython deve converter esses objetos para estruturas C++ imutáveis ou quase imutáveis, preferencialmente sem lógica de negócio. O papel dessa camada é tipar, validar forma básica de dados, administrar referências e liberar o GIL em chamadas bloqueantes. O papel do C++ é possuir os recursos nativos: descritores, `pidfd`, canais de sync, alocação de estruturas de protocolo, estado interno do worker e execução da máquina de estados.

Em termos de API nativa, vale a pena modelar um `LaunchSpec` imutável, um `LaunchHandle` que representa a tentativa em curso e um `LaunchOutcome` que resume o desfecho. O `LaunchHandle` precisa saber devolver `pid`, `pidfd`, o último stage confirmado, o transcript parcial e um método de espera por eventos. Do lado Python, isso pode virar um objeto com interface síncrona e outra assíncrona. A interface assíncrona é útil porque o control plane provavelmente estará em `asyncio` ou em um modelo semelhante de multiplexação. Mesmo nesse caso, a leitura real de `epoll`, `recvmsg`, `read` do pipe e `waitid` deve continuar no domínio nativo ou ser cuidadosamente encapsulada, para que o GIL não se

transforme em gargalo e para que a semântica do protocolo não dependa de callbacks Python em janelas sensíveis.

Há também uma recomendação importante sobre exceções. No C++ de alto nível, usar exceções para propagação de erro de configuração pode ser aceitável; mas a trilha mais íntima entre ``clone`/`fork`` e ``execve()`` deve evitar qualquer dependência de stack unwinding complexo. Nesse trecho, um estilo mais próximo de C robusto costuma ser superior: funções ``noexcept``, retorno de estruturas ``Expected`` ou códigos explícitos, buffers fixos para mensagens e ``write()`` direto no descritor de erro. O Cython deve traduzir apenas o resultado já consolidado pelo pai ou pelo worker, não participar da trilha terminal do filho que está prestes a fazer ``_exit()``.

Um detalhe muitas vezes negligenciado é a propriedade de ownership dos descritores. Em um desenho saudável, Python quase nunca deve ser o dono original do pipe de barreira de ``exec`` nem do socket de stage. Quem cria e possui esses recursos deve ser o worker C++, porque ele entende o ciclo de vida completo e sabe exatamente quando uma ponta precisa ser fechada. Se, por razões de topologia, o Python precisar iniciar o helper e passar descritores, isso deve ocorrer com ``pass_fds`` ou por transferência via socket Unix, nunca por herança implícita. A relação entre ``PEP 446``, ``close_fds=True`` e ``pass_fds`` deve ser tratada como parte do contrato de integração, não como detalhe incidental.

```
# sketch simplificado de fronteira Cython

cdef extern from "runtime_launcher.hpp" namespace "rt":
    cdef cppclass LaunchHandle:
        int pid() noexcept
        int pidfd() noexcept
        bint wait_next_event(...) nogil except +

cpdef object wait_next_event(PyLaunchHandle h):
    cdef Event ev
    with nogil:
        ok = h._native.wait_next_event(ev)
    if not ok:
        raise StopIteration
    return Event.from_native(ev)
```

Figura textual 2 — Exemplo mínimo de fronteira Cython para consumo de eventos do worker nativo.

8. Low Level Design do handshake

O desenho de canais recomendado é simples de enunciar e deliberadamente redundante em semântica. O worker nativo cria um canal de stage por ``socketpair(AF_UNIX, SOCK_SEQPACKET | SOCK_CLOEXEC, 0, ...)``, um canal de barreira de ``exec`` por ``pipe2(O_CLOEXEC)``, um canal separado para logs ou console, e um ``pidfd`` ou mecanismo equivalente de espera robusta pelo processo filho. A criação deve ocorrer antes do ``clone`` ou ``fork``, e ambos os lados devem fechar imediatamente todas as pontas que não usam. Esse fechamento precoce não é um detalhe de limpeza; é parte da correção do protocolo. Se o pai mantiver uma

referência extra para a ponta de escrita do pipe de erro, o EOF nunca chegará e o runtime poderá bloquear indefinidamente.

O canal de stage deve ser tratado como protocolo versionado. Seu trabalho é carregar eventos, pedidos e confirmações. Um evento informa ao pai que o filho chegou a um estágio. Um pedido informa que o filho depende de alguma ação do pai, como mapear ``uid`/`gid``, executar hooks OCI, abrir um mount FD no contexto correto ou encaminhar um descritor de seccomp listener. Uma confirmação autoriza o avanço. A escolha de ``SOCK_SEQPACKET`` é superior à de ``SOCK_STREAM`` porque preserva fronteiras de mensagem, reduz ambiguidade de framing e combina naturalmente com ``SCM_RIGHTS`` para passagem de descritores.

O pipe ``CLOEXEC``, por sua vez, deve ser minimalista. Ele não é o lugar para negociar estágios. Ele é o lugar para uma única semântica forte: o sucesso do ``execve()`` se manifesta como EOF, e o fracasso terminal pré-exec se manifesta como leitura de um frame de erro. O filho não precisa escrever um byte de “sucesso”; o próprio kernel já codifica o sucesso no fechamento do descritor em ``exec``. Escrever um “OK” reintroduz passos desnecessários, amplia janelas de corrida e enfraquece a elegância do contrato. O caminho de sucesso é o silêncio seguido de EOF; o caminho de falha é um frame compacto e estruturado.

Convém distinguir três fronteiras de estado no modelo. A primeira é “processo criado”, que existe logo após ``fork()`` ou ``clone()``. A segunda é “bootstrap consistente”, que corresponde ao estágio ``READY_TO_START`` ou equivalente, no qual o container já está preparado, mas ainda não executou o payload. A terceira é “fronteira de ``exec`` cruzada”, que ocorre quando o pipe ``CLOEXEC`` fecha por sucesso de ``execve()``. A aplicação “estar pronta” é uma quarta fronteira, de domínio da própria aplicação, e não deve ser embutida no handshake do runtime. Essa separação poupa o projeto de um erro recorrente: confundir start de processo com readiness de serviço.

Fluxo de sucesso

Pai / worker	Filho de bootstrap	Processo final
cria socket stage		
cria pipe CLOEXEC		
fork/clone	-----> inicia com write-end aberto	
aguarda events	prepara namespaces/rootfs/cgroups	
<-----	READY_TO_START	
persistir CREATED		
----->	AUTHORIZE_EXEC	
<-----	READY_FOR_EXEC	
	execve(payload)	
<----- EOF no pipe de exec	(FD fecha automaticamente no exec)	
		processo já é a carga útil

Figura textual 3 — Fluxo nominal do handshake com ``create`/`start`` e barreira de ``exec``.

Fluxo de falha terminal pré-exec

```

Pai / worker                                Filho de bootstrap
-----
cria pipe CLOEXEC
fork/clone -----> inicia bootstrap
                                falha em mount / credenciais / execve
                                captura errno imediatamente
                                write(ExecErrorFrame)
<----- frame de erro ----- stage=ROOTFS ou EXECVE, errno=...
                                _exit(127)

marca BOOTSTRAP_FAILED
reap do processo

```

Figura textual 4 — Fluxo terminal de falha pré-exec com frame estruturado.

Exemplo de coordenação rootless / userns

```

Pai / worker                                Filho estágio 1                                Filho final
-----
fork/clone -----> cria user namespace
<----- USERNS_REQUEST
escreve uid_map/gid_map
-----> USERNS_ACK
                                cria processo final / entra em PID ns
<----- CHILD_PID_ANNOUNCE
registra pid final
-----> CHILD_PID_ACK
<----- READY_TO_START / READY_FOR_EXEC
-----> AUTHORIZE_EXEC
<----- EOF no pipe de exec ou frame de erro -----

```

Figura textual 5 — Exemplo de coordenação rootless com pedido de mapeamento ao pai.

```

struct StageHeader {
    uint32_t magic;           // ex.: 0x48534B31 ("HSK1")
    uint16_t version;         // versão do protocolo
    uint16_t msg_type;        // EVENT, REQUEST, ACK, ERROR
    uint16_t stage;           // PREINIT, USERNS, ROOTFS, READY_TO_START...
    uint16_t flags;           // NEEDS_ACK, HAS_FD, TERMINAL...
    uint32_t seq;             // monotônico por sessão
    int32_t errno_value;      // opcional
    uint64_t mono_ns;         // timestamp monotônico
    uint32_t payload_len;     // tamanho do payload
};

struct ExecErrorFrame {
    uint32_t magic;
    uint16_t version;
    uint16_t stage;
    int32_t errno_value;
    int32_t domain_code;
    uint32_t detail_len;
    char    detail[256];     // detalhe curto, redigido e estável
};

```

Figura textual 6 — Exemplo de estruturas mínimas para o protocolo.

Uma forma pragmática de materializar o protocolo é manter o canal de stage com um header binário fixo e payload opcional em JSON ou CBOR. O header deve carregar `magic`, `version`, `message\_type`, `stage`, `seq`, `flags`, `errno\_value`, `mono\_ns` e `payload\_len`. O payload, quando existir, carrega detalhes humanos, parâmetros de hook, informações sobre mount points ou instruções de ack. Em paralelo, o pipe da barreira de `exec` pode usar um frame ainda menor, talvez `magic`, `version`, `stage`, `errno`, `domain\_code` e um texto curto. Separar esses dois formatos é saudável: um protocolo é para coordenação de estágios; o outro é para resolver uma única pergunta binária com forte apoio do kernel.

A versão do protocolo deve ser explícita desde o início, mesmo em um projeto greenfield. Sem esse cuidado, qualquer mudança futura em enumerações, campos ou semânticas de stage quebrará silenciosamente a capacidade de o Python interpretar erros antigos ou de o worker nativo conviver com versões diferentes durante rollout. Como o escopo é local e single-host, o versionamento pode ser simples, mas precisa existir. Também é recomendável que mensagens desconhecidas sejam tratadas como erro estruturado de protocolo e não ignoradas de forma silenciosa, porque ignorar um novo pedido do filho em runtime privilegiado pode deixar o container num estado inconsistente.

A trilha de sucesso precisa ser tratada com a mesma seriedade da trilha de falha. Em particular, o pai deve registrar quando recebeu ``READY_TO_START``, quando enviou ``AUTHORIZE_EXEC``, quando observou ``READY_FOR_EXEC``, quando o pipe de barreira fechou, quando obteve o primeiro ``waitid`` ou sinal de ``pidfd``, e quando persistiu a projeção de estado final. Em termos de observabilidade, isso permite construir métricas como latência de bootstrap por stage, taxa de falha por estágio, distribuição de ``errno``, número de falhas por política de seccomp ou hook OCI, e correlação entre ambientes rootless e rootful. Esse nível de visibilidade é uma das maiores vantagens de modelar o handshake como protocolo e não como detalhe implícito.

Uma implementação madura deve modelar explicitamente a máquina de estados do pai e do filho. No filho, uma sequência razoável é ``SPAWNED``, ``PREINIT``, ``USERNS_REQUESTED`` se aplicável, ``NAMESPACES_CONFIGURED``, ``CGROUPS_ATTACHED``, ``ROOTFS_PREPARED``, ``HOOKS_PENDING``, ``SECCOMP_PENDING``, ``READY_TO_START``, ``READY_FOR_EXEC`` e, finalmente, a tentativa de ``EXECVE``. No pai, a sequência correspondente é ``LAUNCHING``, ``WAITING_STAGE``, ``WAITING_PARENT_ACTION``, ``CREATED``, ``STARTING``, ``WAITING_EXEC_BARRIER``, ``RUNNING``, ``BOOTSTRAP_FAILED`` ou ``EXITED``. Esses nomes podem variar, mas a ideia central é que cada transição tenha um motivo observável e que o control plane só persista estados que correspondam a evidência recebida.

No fluxo com ``create`` e ``start`` separados, o filho envia ``READY_TO_START`` quando terminou a preparação do ambiente e então bloqueia aguardando uma autorização explícita do pai. Quando o control plane chama ``start``, o pai envia um comando como ``RUN`` ou ``AUTHORIZE_EXEC``, o filho continua, opcionalmente emite ``READY_FOR_EXEC``, e então tenta o ``execve()``. Em um fluxo simplificado sem separação entre ``create`` e ``start``, ``READY_TO_START`` e ``READY_FOR_EXEC`` podem colapsar, mas ainda assim a distinção conceitual deve ser preservada no código, porque mais cedo ou mais tarde o runtime precisará de hooks, rootless ou um start assíncrono e a ausência dessa fronteira cobrará caro.

Há uma sutileza importante no tratamento de EOF do pipe ``CLOEXEC``. EOF por si só prova que a ponta de escrita foi fechada; quando o protocolo é bem implementado, isso significa ``execve()`` bem-sucedido. Mas EOF também pode ocorrer se o processo morrer sem escrever nada. Por isso, o projeto não deve depender de EOF isolado como única prova. A solução correta é combinar três sinais: o EOF do pipe, o ``pidfd`` ou ``waitid`` do processo, e a trilha de stages, especialmente ``READY_FOR_EXEC``. Se o pai recebeu ``READY_FOR_EXEC`` e depois observa EOF, a interpretação correta é “exec cruzado”. Se o processo morreu antes disso, sem frame de erro e sem evidência de prontidão para ``exec``, a classificação correta é “morte anômala em bootstrap”, não sucesso.

O frame de erro no pipe de ``exec`` deve ser pequeno, autoexplicativo e estável no tempo. Ele precisa carregar um ``magic`` para validar framing, uma ``version``, o ``stage`` em que o erro ocorreu, o ``errno``, um código de domínio mais amigável para a camada Python e um campo de detalhe textual com tamanho limitado. O importante é que esse frame seja escrito o mais próximo possível da falha e com o menor número possível de alocações ou abstrações de alto nível. O filho deve capturar ``errno`` imediatamente, preencher uma estrutura fixa, fazer ``write()`` diretamente no descritor do pipe e então chamar ``_exit()``. Não se deve confiar em `iostreams`, ``std::cerr``, flush de buffers de ``stdio`` nem handlers complexos na trilha de falha terminal.

No caminho de robustez do filho, vale instalar ``PR_SET_PDEATHSIG`` logo após a criação do processo e verificar se o worker pai continua vivo, evitando órfãos difíceis de diagnosticar quando o lançador morre no meio do bootstrap. Também vale reduzir o conjunto de operações entre a última mensagem ``READY_FOR_EXEC`` e a chamada de ``execve()`` ao estritamente necessário. Quanto menor a janela entre esses dois eventos, menor a chance de morte anômala introduzir ambiguidade residual. Em paralelo, o pai deve multiplexar o socket de stage, o pipe de erro e o ``pidfd`` em uma única espera dirigida por ``epoll`` ou mecanismo equivalente, para que classificação de estados não dependa de polling grosseiro nem de sleeps arbitrários.

Higiene de file descriptors e minimização de privilégio não são detalhes colaterais dessa feature; eles fazem parte do seu valor. Todo descritor criado para o handshake deve nascer com ``CLOEXEC``. Todo descritor que precise sobreviver ao ``exec`` do payload, como ``stdin``, ``stdout``, ``stderr``, PTY, socket de console ou outro FD explicitamente passado ao processo do container, deve ser listado de forma positiva e intencional. Em C++, isso pede wrappers RAII para ``UniqueFd``, ``SocketPair``, ``PipePair`` e ``PidFd``, além de uma rotina de saneamento que use ``close_range`` quando disponível ou fallback equivalente. A meta é simples: o payload só enxerga o que foi deliberadamente exposto.

No caminho de segurança, também vale separar o que é erro estruturado do que é dado sensível. O transcript de stages é útil demais para ser descartado, mas não deve vaziar variáveis de ambiente, segredos montados em arquivos, conteúdo integral de specs ou paths internos que comprometam o host. O worker nativo deve aplicar políticas de redaction antes de anexar detalhes textuais aos erros. Isso é especialmente importante quando o control plane expõe `inspect` por API multi-tenant ou persiste transcripts em armazenamento compartilhado.

Existem alternativas de desenho que parecem mais simples, mas acumulam dívida técnica rapidamente. A primeira é usar somente o status de saída do processo filho. Essa abordagem falha porque não distingue falha pré-exec de saída rápida pós-exec e não preserva stage nem `errno`. A segunda é usar apenas `stderr` como canal semântico. Ela falha porque mistura saída do launcher, de hooks, do loader e da própria aplicação, além de depender de parsing frágil de texto. A terceira é usar um único `SOCK_STREAM` bidirecional para tudo. Essa opção até pode funcionar, mas exige framing mais delicado, dificulta a evolução do protocolo e perde a clareza conceitual de ter uma barreira de `exec` minimalista e um canal de stage rico. A quarta é colocar a lógica pós-fork em callbacks Python. Essa, para o contexto do seu projeto, deve ser evitada de forma categórica.

Outro risco comum é supercarregar o handshake com semântica de health check. O runtime pode provar que o kernel carregou o binário e que o processo já vive no ambiente correto; ele não pode provar, pelo mesmo mecanismo, que uma aplicação HTTP terminou sua inicialização, que uma migração de banco acabou ou que uma porta está escutando com sucesso. Quando se mistura essas camadas, o protocolo de bootstrap deixa de ser confiável, porque passa a carregar timeouts arbitrários e suposições específicas de aplicação. A regra de projeto deve ser simples: handshake de runtime mede bootstrap e `exec`; readiness de aplicação mede comportamento funcional depois disso.

Há ainda um risco operacional ligado à evolução do protocolo. Quando novos stages entram em cena, como suporte a seccomp listener, mount FD, time namespaces ou integrações futuras, a tendência natural de uma base de código é abrir exceções ad hoc no fluxo. Esse é o momento em que a solução sem máquina de estados degrada rapidamente. Por isso, versionar mensagens, manter códigos de stage enumerados, registrar transcript monotônico e centralizar a lógica de transição em um objeto de estado não são luxos. Eles são a diferença entre um runtime que cresce com consistência e um runtime que vira uma coleção de ifs acoplados a detalhes de kernel.

Sinal observado pelo pai	Interpretação correta	Estado projetado
Frame de erro no pipe de exec	Falha explícita de bootstrap ou <code>execve</code>	BOOTSTRAP_FAILED

Sinal observado pelo pai	Interpretação correta	Estado projetado
EOF no pipe + READY_FOR_EXEC ou processo ainda vivo	Fronteira de exec cruzada	RUNNING / STARTED
Morte do processo sem frame e sem READY_FOR_EXEC	Morte anômala antes de exec	BOOTSTRAP_FAILED_INTERNAL
Exec cruzado e saída imediata depois	Start bem-sucedido; aplicação saiu	EXITED (com started_at válido)

Tabela 4 — Classificação recomendada para sinais concorrentes observados pelo pai.

9. Domain Driven Design

Do ponto de vista de Domain Driven Design, essa feature deixa de ser “só um IPC” quando o sistema passa a tratar tentativa de start como um agregado próprio. O bounded context de controle de ciclo de vida do container, em Python, deve enxergar entidades como ``Container``, ``LaunchAttempt`` e ``ContainerProcessProjection``. O bounded context de runtime, em C++, deve enxergar entidades como ``BootstrapSession``, ``StageMessage``, ``ExecBarrierOutcome`` e ``ProcessHandle``. Entre eles, o Cython atua como anti-corruption layer: ele traduz estruturas e erros do mundo nativo para o vocabulário do domínio Python sem deixar que detalhes de ABI ou de syscall contaminem a API de alto nível.

O vocabulário ubíquo precisa ser extremamente preciso, porque muitos bugs de runtime são, na verdade, bugs de linguagem. “Criado” não é “rodando”. “Pronto para iniciar” não é “`exec` concluído”. “Iniciou” não é “aplicação pronta”. “Falha de bootstrap” não é “aplicação saiu com erro”. “Morte anômala antes de `exec`” não é “`execve()` falhou com `ENOENT`”. Se o sistema nomeia mal esses estados, o modelamento posterior em banco, API e observabilidade também ficará errado. O handshake proposto é valioso justamente porque produz fatos suficientes para sustentar esse vocabulário com evidência de kernel.

A principal invariável de domínio aqui é simples e poderosa: nenhum ``LaunchAttempt`` pode transicionar para sucesso enquanto a barreira de ``exec`` não tiver sido resolvida. Outra invariável útil é que a sequência de stage events deve ser monotônica e compatível com a máquina de estados. Uma terceira invariável separa a responsabilidade do runtime da responsabilidade da aplicação: o máximo que o handshake do runtime pode afirmar é que o processo foi criado, preparado e, eventualmente, que a imagem do programa foi carregada. Qualquer noção de health check, porta escutando ou prontidão funcional deve viver em outro mecanismo.

10. Padrões de Design aplicáveis

Entre os padrões GoF, o mais natural para essa feature é State. Tanto o worker nativo quanto a camada de domínio em Python se beneficiam de uma modelagem explícita da máquina de estados, porque o comportamento permitido em cada ponto muda radicalmente. Em ``WAITING_USERNS_ACK``, por exemplo, o pai deve aceitar apenas a confirmação correta e recusar ``RUN``. Em ``READY_TO_START``, ele pode persistir o container como ``CREATED``, mas ainda não como ``RUNNING``. Em ``WAITING_EXEC_BARRIER``, uma leitura de frame de erro significa uma transição completamente diferente de um EOF seguido de processo vivo.

Strategy também é importante porque o caminho exato de bootstrap varia com rootless versus rootful, cgroup v1 versus v2, necessidade ou não de hooks OCI, presença de seccomp listener e assim por diante. O algoritmo macro de bootstrap permanece o mesmo, mas certos estágios são inseridos, removidos ou especializados por estratégia. Template Method ajuda a organizar isso no C++: a sequência geral “preparar, pedir ações ao pai quando necessário, sinalizar prontidão, autorizar ``exec``, cruzar barreira de ``exec``” continua fixa, e cada subtipo ou política substitui apenas os passos específicos.

Bridge, Adapter e Facade resolvem a integração entre linguagens. O Python deve enxergar uma fachada simples, algo como ``create_container()``, ``start_container()``, ``wait_launch_attempt()``, ``inspect_transcript()``. Por baixo dessa fachada, o Cython adapta tipos Python para tipos nativos e expõe uma bridge para diferentes backends de runtime, como um worker local, um daemon nativo ou até, no futuro, um backend remoto. Observer encaixa bem na distribuição de stage events para logs, métricas, tracing e persistência. Command aparece no protocolo do canal de stage, em mensagens como ``RUN_HOOKS``, ``AUTHORIZE_EXEC``, ``MAP_USERNS`` ou ``FORWARD_SECCOMP_FD``. O valor desses padrões não é acadêmico; eles ajudam a impedir que o handshake vire um bloco procedural opaco e difícil de evoluir.

11. Requirements funcionais

Em termos funcionais, a implementação deve oferecer uma semântica inequívoca para as transições de ``create`` e ``start``. Ela deve distinguir, para cada tentativa de launch, o instante em que o processo filho foi criado, o instante em que o bootstrap alcançou um ponto consistente, e o instante em que o ``execve()`` da carga útil foi efetivamente bem-sucedido. Ela deve ser capaz de transportar erros estruturados contendo stage, ``errno``, contexto textual e um código de domínio estável, para que a camada Python traduza a falha em mensagens de API compreensíveis e em estado persistido correto. Ela deve suportar mensagens por stage com possibilidade de ACK, timeout, retransmissão logicamente idempotente do ponto de vista do pai e, quando necessário, passagem de file descriptors. Ela deve suportar pelo menos o caminho rootful simples e ser desenhada desde o início para acomodar rootless, hooks OCI, seccomp listener e eventual separação formal entre ``create`` e ``start``.

A implementação também deve tornar observável a história do bootstrap. Isso significa persistir um transcript ou projeção equivalente com timestamps monotônicos, sequência de messages, PID final, eventual ``pidfd``, resultado da barreira de ``exec``, causa terminal de falha e relação com o ``container_id`` e o ``launch_attempt_id``. O sistema deve permitir que ``inspect`` ou endpoint semelhante exponha essas informações sem obrigar o operador a depender de logs de debug. Finalmente, a API de alto nível deve refletir a semântica real do runtime: ``start`` deve falhar quando a barreira de ``exec`` falhar, deve poder retornar sucesso mesmo se o processo sair logo em seguida após um ``exec`` bem-sucedido, e nunca deve confundir um erro de bootstrap com um exit code da aplicação já iniciada.

12. Requirements não funcionais

Em termos não funcionais, o requisito dominante é determinismo operacional. O protocolo precisa ser livre de ambiguidades habituais de concorrência, como vazamento de descritores que impede EOF, writes parciais mal interpretados, reuso de PIDs sem ``pidfd``, inversão de ordem entre observação de morte do processo e leitura do pipe, ou travamentos por depender de callbacks Python em janela pós-fork. Segurança vem logo em seguida: nenhum descritor não explicitamente permitido deve atravessar o ``exec``; toda a lógica de herança deve ser positiva e não implícita; o transcript não deve vaziar segredos; e o caminho rootless deve respeitar a ordem correta de ``setgroups``, ``uid_map`` e ``gid_map``.

Desempenho também importa, mas deve ser tratado como restrição de segunda ordem. O handshake é curto em volume de dados; sua relevância vem da semântica, não da largura de banda. Ainda assim, a implementação deve evitar alocações desnecessárias na trilha crítica, liberar o GIL em operações bloqueantes do lado Cython, usar ``epoll`` ou mecanismo equivalente para multiplexar canais sem busy waiting e preferir estruturas RAII no C++ para garantir fechamento confiável de recursos. Testabilidade e evolutividade completam os requisitos não funcionais. O protocolo precisa ser versionado, fácil de simular em testes, tolerante à extensão de novos stages, e acompanhado de métricas e tracing suficientes para depuração de produção.

13. Plano de implementação incremental

A melhor estratégia de implementação é incremental. Na primeira fase, vale construir o mínimo correto: um worker nativo em C++ que cria o filho, instala um ``pipe2(O_CLOEXEC)`` dedicado, tenta um bootstrap simplificado e escreve um frame estruturado de erro na trilha terminal de falha. Nessa fase, o objetivo não é suportar todos os features de container, e sim fechar a ambiguidade central entre “não conseguiu executar” e “executou e depois saiu”. Já aqui o Python deve receber um ``LaunchAttempt`` persistível e o transcript mínimo com ``spawned_at``,

``exec_barrier_resolved_at`` e eventual erro terminal.

Na segunda fase, entra o canal de stage por ``SOCK_SEQPACKET``. É o momento de separar ``READY_TO_START`` de ``READY_FOR_EXEC``, implementar um protocolo de requests e ACKs e alinhar a semântica com ``create`` e ``start`` em estilo OCI. Também é nessa fase que o transcript deixa de ser um apêndice técnico e vira um artefato de domínio. O Python passa a enxergar estágios explícitos, e o Cython passa a traduzir esses eventos com tipos estáveis.

Na terceira fase, o runtime deve absorver cenários em que o handshake é realmente necessário para coordenação com o pai, e não apenas para observabilidade. Rootless com ``usersns`` é o primeiro grande exemplo, porque a escrita de ``uid_map`` e ``gid_map`` exige ordem estrita. Em seguida entram hooks OCI, passagem de mount FDs e cenários de seccomp listener. Aqui a proximidade conceitual com ``runc`` fica mais forte, embora a implementação permaneça idiomática em C++.

Na quarta fase, a preocupação muda de correção semântica para endurecimento operacional. Entram ``pidfd`` onde disponível, fallback compatível quando não houver, ``close_range`` para higiene massiva de descritores, métricas por stage, tracing distribuído, testes de estresse e validação de compatibilidade com kernels diferentes. Só depois disso faz sentido otimizar microdetalhes de cópia ou serialização. Em runtimes de containers, o custo de um start mal classificado é muito maior do que o custo de alguns microssegundos extras de IPC.

14. Estratégia de testes e validação

A estratégia de testes precisa ser tão estruturada quanto o próprio protocolo. Testes unitários devem cobrir framing das mensagens, serialização do frame de erro, comportamento de RAII para fechamento de descritores, preservação correta de ``errno`` e tradução de exceções C++ para erros Python. O canal de stage deve ter testes de máquina de estados que rejeitem sequências inválidas, como ``RUN`` antes de ``READY_TO_START`` ou ``HOOKS_DONE`` sem ``HOOKS_REQUEST``. Também vale ter testes específicos para versionamento de protocolo, garantindo que campos novos possam ser ignorados ou negociados sem quebrar backward compatibility local.

Nos testes de integração, é preciso induzir falhas reais de bootstrap e não apenas mocks. O conjunto mínimo inclui binário inexistente (``ENOENT``), binário sem permissão de execução (``EACCES``), formato inválido (``ENOEXEC``), diretório de trabalho inválido, falha de mount da rootfs, erro de cgroup, seccomp mal configurado, hook OCI que retorna erro e tentativa rootless sem mapas válidos. Para cada cenário, o teste deve verificar o stage terminal, o ``errno``, a mensagem de domínio exposta ao Python, o estado persistido no control plane e a ausência de

marcação incorreta do container como ``RUNNING``.

Testes de estresse e caos são indispensáveis porque muitas falhas desse tipo só aparecem na presença de concorrência. O sistema deve ser exercitado com centenas ou milhares de starts rápidos, inclusive payloads que executam e saem quase instantaneamente. É nesse regime que aparecem bugs de retenção indevida da ponta de escrita do pipe, ordenações erradas entre EOF e ``waitid``, vazamento de FDs e uso incorreto de descritores herdáveis. Também vale injetar mortes anômalas do bootstrap antes de ``exec``, para garantir que EOF sem stage adequado não seja interpretado como sucesso.

Por fim, os critérios de aceite devem refletir semântica e não apenas “testes verdes”. O sistema está correto quando consegue provar, em produção, a diferença entre falha de bootstrap, sucesso de ``exec`` seguido de saída rápida da aplicação e prontidão funcional da aplicação em camada superior. Quando um operador inspeciona um launch, ele deve conseguir responder, sem adivinhação, “o processo foi criado?”, “o ambiente do container ficou consistente?”, “o ``exec`` foi carregado?”, “em que stage falhou?” e “que ação do pai estava pendente?”.

15. Conclusão

A síntese do problema é objetiva. O recurso pedido não é apenas um “pipe para saber se deu certo”. Ele é a formalização de uma fronteira de responsabilidade dentro do runtime. Se o seu sistema quer se aproximar da robustez conceitual do Docker original, precisa separar criação de processo, sucesso de bootstrap, sucesso de ``exec`` e prontidão da aplicação. Um ``pipe2(O_CLOEXEC)`` dedicado é a melhor ferramenta para a barreira final de ``exec``; um ``socketpair(AF_UNIX, SOCK_SEQPACKET | SOCK_CLOEXEC)`` é a melhor ferramenta para mensagens por stage, ACKs e passagem de FDs. Juntos, esses canais produzem uma semântica que o control plane consegue persistir e expor sem inferência frágil.

Em termos práticos, a recomendação final é implementar o handshake no domínio nativo em C++, usar Cython apenas como ponte tipada e preservar no Python a responsabilidade por metadados, API e projeções de estado. Essa solução respeita o desenho de camadas que fez Docker, containerd e ``runc`` funcionarem bem: políticas e domínio acima; syscalls, bootstrap e relação íntima com o kernel abaixo. O custo extra de projetar bem esse handshake no início é pequeno. O custo de tentar “remendar depois” um runtime que não sabe, com precisão, se falhou antes ou depois do ``exec`` é muito maior.

16. Referências

1. Docker, “Docker overview”. URL: <https://docs.docker.com/get-started/docker-overview/> . Acesso em 12 mar. 2026.

2. Docker, “Alternative container runtimes”. URL: <https://docs.docker.com/engine/daemon/alternative-runtimes/> . Acesso em 12 mar. 2026.
3. containerd, “Runtime v2 README”. URL: <https://github.com/containerd/containerd/blob/main/core/runtime/v2/README.md> . Acesso em 12 mar. 2026.
4. Open Container Initiative, “OCI Runtime Specification — runtime.md”. URL: <https://github.com/opencontainers/runtime-spec/blob/main/runtime.md> . Acesso em 12 mar. 2026.
5. Open Container Initiative, “OCI Runtime Specification — config.md”. URL: <https://github.com/opencontainers/runtime-spec/blob/main/config.md> . Acesso em 12 mar. 2026.
6. Linux man-pages project, “open(2)”. URL: <https://man7.org/linux/man-pages/man2/open.2.html> . Acesso em 12 mar. 2026.
7. Linux man-pages project, “pipe(2)”. URL: <https://man7.org/linux/man-pages/man2/pipe.2.html> . Acesso em 12 mar. 2026.
8. Linux man-pages project, “execve(2)”. URL: <https://man7.org/linux/man-pages/man2/execve.2.html> . Acesso em 12 mar. 2026.
9. Linux man-pages project, “F_GETFD(2const) / FD_CLOEXEC”. URL: https://man7.org/linux/man-pages/man2/F_GETFD.2const.html . Acesso em 12 mar. 2026.
10. Linux man-pages project, “pidfd_open(2)”. URL: https://man7.org/linux/man-pages/man2/pidfd_open.2.html . Acesso em 12 mar. 2026.
11. Linux man-pages project, “close_range(2)”. URL: https://man7.org/linux/man-pages/man2/close_range.2.html . Acesso em 12 mar. 2026.
12. Linux man-pages project, “clone(2)”. URL: <https://man7.org/linux/man-pages/man2/clone.2.html> . Acesso em 12 mar. 2026.
13. Python Software Foundation, “PEP 446 – Make newly created file descriptors non-inheritable”. URL: <https://peps.python.org/pep-0446/> . Acesso em 12 mar. 2026.
14. Python Software Foundation, “os — Miscellaneous operating system interfaces”. URL: <https://docs.python.org/3/library/os.html> . Acesso em 12 mar. 2026.
15. Python Software Foundation, “subprocess — Subprocess management”. URL: <https://docs.python.org/3/library/subprocess.html> . Acesso em 12 mar. 2026.
16. Python Software Foundation, “What’s New In Python 3.3” (inclui `os.pipe2`). URL: <https://docs.python.org/3/whatsnew/3.3.html> . Acesso em 12 mar. 2026.
17. Cython Project, “Wrapping C++”. URL: https://docs.cython.org/en/latest/src/userguide/wrapping_CPlusPlus.html . Acesso em 12 mar. 2026.
18. Cython Project, “Extension types / cdef classes”. URL: https://cython.readthedocs.io/en/latest/src/tutorial/cdef_classes.html . Acesso em 12 mar. 2026.
19. Cython Project, “Cython and the GIL”. URL: <https://cython.readthedocs.io/en/latest/src/userguide/nogil.html> . Acesso em 12 mar. 2026.
20. opencontainers/runc, “libcontainer/process_linux.go”. URL: https://raw.githubusercontent.com/opencontainers/runc/main/libcontainer/process_linux.go . Acesso em 12 mar. 2026.
21. opencontainers/runc, “libcontainer/init_linux.go”. URL: https://raw.githubusercontent.com/opencontainers/runc/main/libcontainer/init_linux.go . Acesso em 12 mar. 2026.
22. opencontainers/runc, “libcontainer/sync.go”. URL: <https://raw.githubusercontent.com/opencontainers/runc/main/libcontainer/sync.go> . Acesso em 12 mar. 2026.
23. opencontainers/runc, “libcontainer/sync_unix.go”. URL: https://raw.githubusercontent.com/opencontainers/runc/main/libcontainer/sync_unix.go . Acesso em 12 mar. 2026.

24. opencontainers/runc, "libcontainer/utils/utils_unix.go". URL:
https://raw.githubusercontent.com/opencontainers/runc/main/libcontainer/utils/utils_unix.go .
Acesso em 12 mar. 2026.
25. opencontainers/runc, "libcontainer/nsenter/nsexec.c". URL:
<https://raw.githubusercontent.com/opencontainers/runc/main/libcontainer/nsenter/nsexec.c> .
Acesso em 12 mar. 2026.
26. opencontainers/runc, "Issue #5087 — sync socket EOF / close semantics". URL:
<https://github.com/opencontainers/runc/issues/5087> . Acesso em 12 mar. 2026.